Calculating Data Quality Dimensions with Spark

In this train, we'll get acquainted with essential tools for maintaining high data quality standards. Specifically, we'll learn how to use Spark to calculate each of the six dimensions of data quality through practical examples and hands-on exercises.

Learning Objectives

By the end of this train, you should be able to:

- Use Spark to calculate each of the six dimensions of data quality.
- Understand how each dimension affects the trustworthiness and usability of your data.
- Apply practical Spark transformations and aggregations to measure and improve data quality.

Measuring Data Quality Dimensions in Spark

1. Completeness

Completeness measures the presence of expected values in a dataset.

```
from pyspark.sql.functions import col, count, when
#Count missing values in the 'age' column
df.select([count(when(col('age').isNull(), 1)).alias('missing_age')]).show()
```

2. Uniqueness

Uniqueness ensures each record or identifier appears only once where expected.

```
#Check for duplicate customer IDs
df.groupBy("customer_id").count().filter("count > 1").show()
```

3. Validity

Validity ensures values adhere to domain constraints or formats (e.g., valid email, age in range).

```
# Validate that age is between 0 and 120
df.filter((col("age") < 0) | (col("age") > 120)).show()
```

4. Accuracy

Accuracy reflects how close data is to the real-world values it's meant to represent. This usually requires cross-reference.

```
# Example: Comparing with a reference DataFrame
df.join(reference_df, "id").filter(df["postal_code"] !=
reference_df["postal_code"]).show()
```

5. Consistency

Consistency ensures data does not conflict across sources or time.

```
# Example: Conflicting values for a customer across systems
df1.join(df2, "customer_id").filter(df1["email"] != df2["email"]).show()
```

6. Timeliness

Timeliness checks if data is delivered or updated within an acceptable timeframe.

```
from pyspark.sql.functions import current_date, datediff

#Flag records older than 30 days
df.withColumn("stale_data", datediff(current_date(), col("last_updated")) >
30).show()
```

Tools for Auditing Data Quality

- **PyDeequ:** A data quality validation tool built on top of Spark. It allows defining constraints programmatically.
- **Spark SQL:** Enables expressing validation rules as SQL queries.
- **Profiling:** Use `df.describe()` or `summary()` to detect anomalies.

Conclusion

Data quality is foundational to building trustworthy data pipelines. By using Spark's distributed processing capabilities, you can efficiently measure and validate quality across massive datasets. Embedding quality checks early in the pipeline helps detect issues before they propagate to downstream systems.

▼Key Takeaways ✖

- **Takeaway 1:** Completeness, uniqueness, validity, accuracy, consistency, and timeliness are the six key dimensions of data quality.
- **Takeaway 2:** Spark makes it easy to compute these dimensions at scale with built-in functions.
- **Takeaway 3:** You can integrate validation checks using PySpark or PyDeequ to automate auditing.
- **Takeaway 4:** Early detection of quality issues reduces costs and improves data trust.
- **Takeaway 5:** Profiling and comparing datasets help identify anomalies and inconsistencies.